TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

Lab 02

Mật mã học hiện đại

Modern Cryptography

Môn học: **An toàn mạng máy tính**

GVTH: **KS.Nguyễn Ngọc Trưởng**

1 Giới thiệu chung

1.1 Mục tiêu

Mục tiêu học tập của bài thực hành này là giúp sinh viên làm quen với các khái niệm trong mật mã học hiện đại, đặc biệt là về các hệ mật mã khối (DES và AES) và mật mã hóa khóa công khai (RSA). Sau khi hoàn thành bài thực hành, sinh viên sẽ có được kinh nghiệm trực quan với các thuật toán mã hóa DES, các chế độ mã hóa, kỹ thuật đệm (padding), vector khởi tạo (Initial Vector - IV) và thuật toán RSA. Bài thực hành này sẽ bao gồm các chủ đề sau:

1. Mã hóa DES, AES.
2. Chế độ hoạt động (Mode of operation) và kỹ thuật đệm (padding).
3. Hiệu ứng thác đổ (Avalanche effect) và độ mạnh của mật mã DES.
4. Mã hóa RSA.
5. Hệ mật mã khóa công khai cho tính bảo mật (Confidentiality) và xác thực (Authentication)

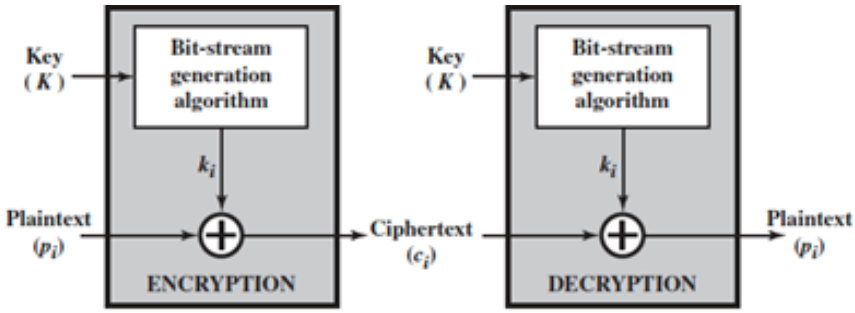
1.2 Kiến thức nền tảng

1.2.1 Mật mã khối: DES/AES

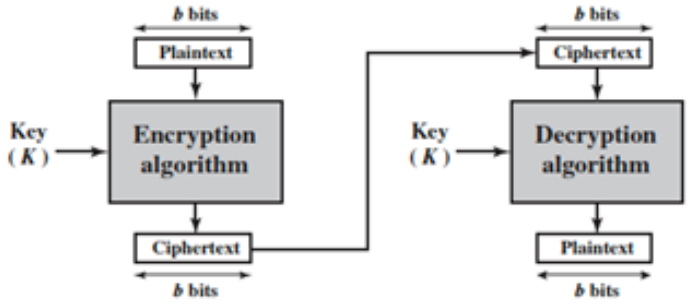
Định lý 1.1: Stream Cipher và Block Cipher

Stream Cipher *(Mật mã luồng)* là một phương pháp mã hóa một dòng dữ liệu số theo từng bit hoặc từng byte. Ví dụ: mật mã Vigenère tự động khóa (autokeyed Vigenère cipher) và mật mã Vernam. Trong trường hợp lý tưởng, một phiên bản của mật mã Vernam sử dụng one-time pad sẽ được dùng, trong đó dòng khóa (keystream - k_i) có độ dài bằng với dòng bit bản rõ (p_i). Nếu dòng khóa mật mã là ngẫu nhiên, hệ mật mã này sẽ không thể bị phá vỡ bằng bất kỳ phương pháp nào ngoại trừ việc có được dòng khóa đó (đạt được tính bảo mật hoàn hảo - perfect secrecy). Tuy nhiên, dòng khóa phải được cung cấp trước cho cả hai người dùng thông qua một kênh độc lập và an toàn.

Block Cipher *(Mật mã khối)* là một phương pháp trong đó một khối bản rõ (block of plaintext) được xem như một thể thống nhất và được sử dụng để tạo ra một khối bản mã (ciphertext block) có cùng độ dài. Thông thường, kích thước khối là 64 hoặc 128 bit được sử dụng. Tương tự như mật mã luồng, hai người dùng chia sẻ một khóa mã hóa đối xứng. Bằng cách sử dụng một số chế độ hoạt động (modes of operation), một mật mã khối có thể được dùng để đạt được hiệu quả tương tự như một mật mã luồng.



(a) Stream cipher using algorithmic bit-stream generator



(b) Block cipher

Hình 1: Stream cypher và Block cypher

Định lý 1.2: Cấu trúc mật mã Feistel

Feistel đề xuất rằng chúng ta có thể xấp xỉ một mật mã khối lý tưởng bằng cách sử dụng khái niệm mật mã tích (product cipher), tức là luân phiên các phép thế (substitutions) và hoán vị (permutations). Điều này có nghĩa là thực hiện hai hoặc nhiều hệ mật mã đơn giản một cách tuần tự sao cho kết quả cuối cùng có độ mạnh mật mã cao hơn bất kỳ thành phần nào. Cấu trúc mật mã Feistel được sử dụng bởi DES và một số hệ mật mã khối đối xứng quan trọng hiện đang được sử dụng (ví dụ: TDEA, TLS).

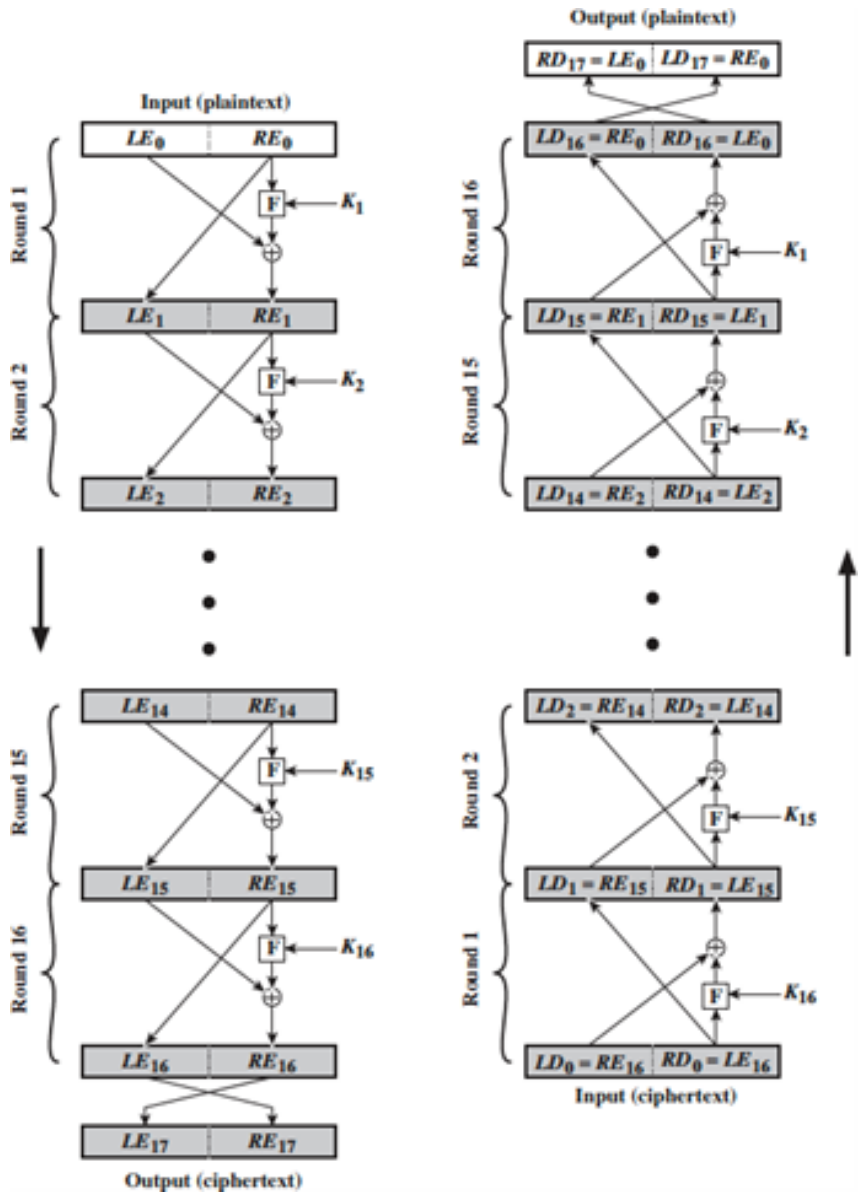
Theo Feistel, trong quá trình mã hóa, bản rõ sẽ được chia thành hai nửa là LE_0 và RE_0. Sau đó, chúng sẽ được xử lý qua n vòng (tất cả các vòng đều có cấu trúc giống nhau, sử dụng các khóa con - subkeys K_i khác nhau). Trong mỗi vòng, nửa bên trái và nửa bên phải được sửa đổi lần lược như sau:

$$LE_i = RE_{i-1} \tag{1}$$

$$RE_i = LE_{i-1} \oplus F(RE_{i-1},\ K_i) \tag{2}$$

Trong đó: K_i là khóa con ở vòng i được tạo ra từ khóa gốc K, và F là một hàm giả ngẫu

nhiên (pseudorandom function) được tham số hóa bởi khóa K_i . Cuối cùng, hai nửa được hoán đổi (phép hoán vị).

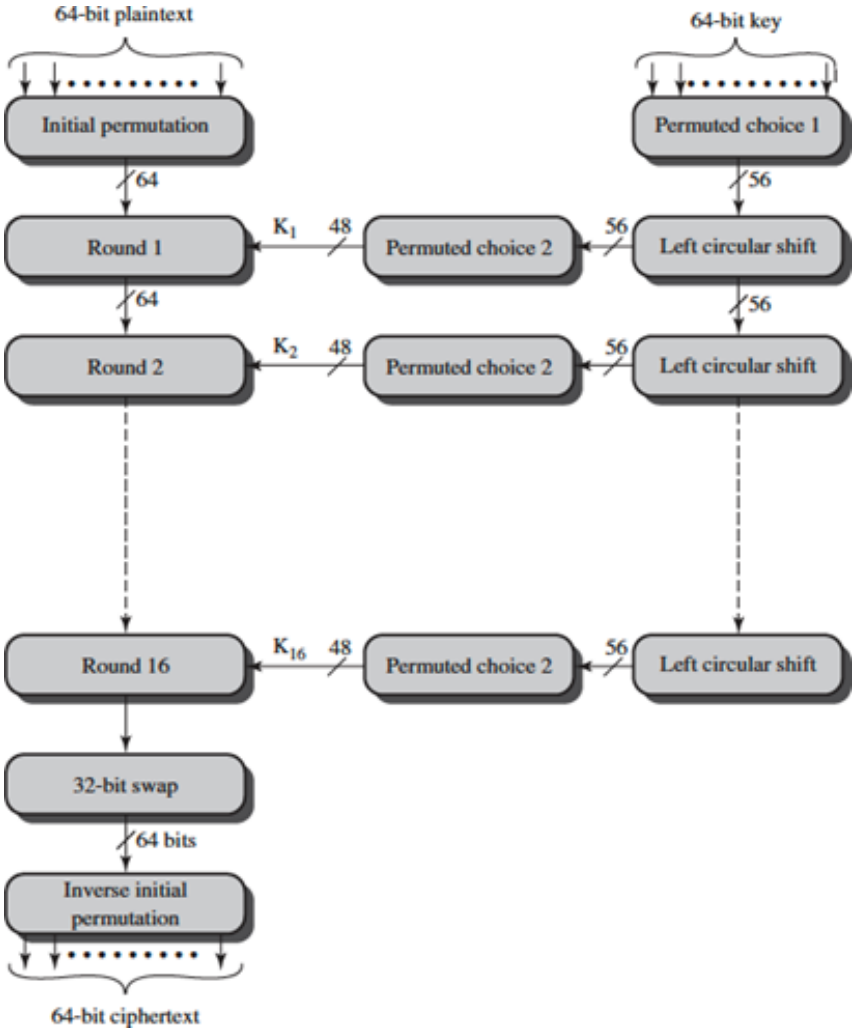


Hình 2: Feistel Cypher Structure (16 rounds)

Định lý 1.3: Data Encryption Standard - DES

DES (Data Encryption Standard) là một thuật toán khóa đối xứng dùng để mã hóa dữ liệu số. Nó được phát triển vào đầu những năm 1970 tại IBM và dựa trên cấu trúc mật mã Feistel. DES được Viện Tiêu chuẩn và Công nghệ Quốc gia (NIST) phê duyệt làm tiêu chuẩn vào năm 1976 và được công bố là một Chuẩn Xử lý Thông tin Liên bang (FIPS) chính thức cho Hoa Kỳ vào năm 1977. DES được sử dụng rộng rãi trên toàn thế giới cho đến năm 1999. NIST đã giới thiệu một tiêu chuẩn mới sử dụng 3-DES, và sau đó nó đã được thay thế bằng AES vào năm 2001.

Mô tả tổng quan về mã hóa DES được trình bày trong Hình 3, với khối bản rõ phải có độ dài 64 bit và khóa có độ dài 56 bit.



Hình 3: General Depiction of DES Encryption Algorithm

Định lý 1.4: Mode of operation

Một mật mã khối nhận đầu vào là một khối văn bản có độ dài cố định bit và một khóa, sau đó tạo ra một khối bản mã dài bit. Nếu lượng bản rõ cần mã hóa lớn hơn bit, mật mã khối vẫn có thể được sử dụng bằng cách chia bản rõ thành các khối bit. Khi nhiều khối bản rõ được mã hóa bằng cùng một khóa, một số vấn đề bảo mật sẽ phát sinh.

Chế độ hoạt động là một kỹ thuật để tăng cường hiệu quả của một thuật toán mật mã hoặc điều chỉnh thuật toán cho một ứng dụng cụ thể, chẳng hạn như áp dụng một mật mã khối cho một chuỗi các khối dữ liệu hoặc một luồng dữ liệu. Năm chế độ hoạt động đã được NIST định nghĩa (trong SP 800-38A) nhằm bao quát nhiều

ứng dụng mã hóa khác nhau mà một mật mã khối có thể được sử dụng. Các chế độ này bao gồm:

- ECB - Electronic Code Book
- CBC - Cypher Block Chaining
- CFB - Cypher Feedback
- OFB - Output Feedback
- CTR - Counter

Mode	Description	Typical Application
Electronic Codebook (ECB)	Each block of plaintext bits is encoded independently using the same key.	• Secure transmission of single values (e.g., an encryption key)
Cipher Block Chaining (CBC)	The input to the encryption algorithm is the XOR of the next block of plaintext and the preceding block of ciphertext.	• General-purpose block-oriented transmission • Authentication
Cipher Feedback (CFB)	Input is processed *s* bits at a time. Preceding ciphertext is used as input to the encryption algorithm to produce pseudorandom output, which is XORed with plaintext to produce next unit of ciphertext.	• General-purpose stream-oriented transmission • Authentication
Output Feedback (OFB)	Similar to CFB, except that the input to the encryption algorithm is the preceding encryption output, and full blocks are used.	• Stream-oriented transmission over noisy channel (e.g., satellite communication)
Counter (CTR)	Each block of plaintext is XORed with an encrypted counter. The counter is incremented for each subsequent block.	• General-purpose block-oriented transmission • Useful for high-speed requirements

Hình 4: Block Cypher Modes of Operation

1.2.2 Mật mã hóa khóa công khai – RSA

Mã hóa đối xứng đã phát triển từ cổ điển đến hiện đại nhưng vẫn còn nhiều hạn chế. Hai trong số những vấn đề khó khăn nhất liên quan đến mã hóa đối xứng là:

- **Key distribution** *(Phân phối khóa)* - Vấn đề trao đổi khóa bí mật giữa bên gửi và bên nhận. Phân phối khóa trong mã hóa đối xứng yêu cầu hoặc (1) hai bên liên lạc đã chia sẻ một khóa từ trước, được phân phối bằng một cách nào đó; hoặc (2) sử dụng một trung tâm phân phối khóa. Cần có một kênh an toàn để trao đổi khóa sao cho khóa phải được giữ bí mật và chỉ bên gửi và bên nhận biết. Việc này sẽ khó thực hiện và việc thiết lập một kênh an toàn như vậy sẽ tốn kém và mất thời gian.

- **Digital signatures** *(Chữ ký số)*: Nếu việc sử dụng mật mã trở nên phổ biến, không chỉ trong các tình huống quân sự mà còn cho các mục đích thương mại và cá nhân, thì các thông điệp và tài liệu điện tử sẽ cần một thứ tương đương với chữ ký được sử dụng trong tài liệu giấy. Hơn nữa, không có cơ sở để quy trách nhiệm nếu khóa bị lộ.

Diffie và Hellman đã đạt được một bước đột phá đáng kinh ngạc vào năm 1976 bằng cách đề xuất một phương pháp giải quyết cả hai vấn đề trên và hoàn toàn khác biệt so với tất cả các phương pháp tiếp cận mật mã trước đó trong hơn bốn thiên niên kỷ. Đó chính là **mật mã hóa khóa công khai** hay **mật mã hóa bất đối xứng**. Sự phát triển của mật mã khóa công khai là cuộc cách mạng lớn nhất và có lẽ là duy nhất trong toàn bộ lịch sử của ngành mật mã học.

Để phân biệt giữa hai hệ mật mã hiện đại (đối xứng và bất dối xứng), chúng ta gọi khóa được sử dụng trong mã hóa đối xứng là khóa bí mật (secret key). Hai khóa được sử dụng cho mã hóa bất đối xứng được gọi là khóa công khai (public key - PU) và khóa riêng tư (private key - PR). Bản rõ được ký hiệu là M, bản mã được ký hiệu là C. Có 2 hướng tiếp cận chính trong mật mã hóa khóa công khai:

- Hệ mật mã khóa công khai cho tính bảo mật (**Confidentiality**): Nếu Bob muốn gửi một tin nhắn bảo mật cho Alice, Bob sẽ mã hóa tin nhắn bằng khóa công khai của Alice. Khi Alice nhận được tin nhắn, cô ấy sẽ giải mã nó bằng khóa riêng tư của mình. Không có người nhận nào khác có thể giải mã tin nhắn vì chỉ Alice mới biết khóa riêng tư của cô ấy.
  - Mã hóa: C = E(M, PU)
  - Giải mã: M = D(C, PR)

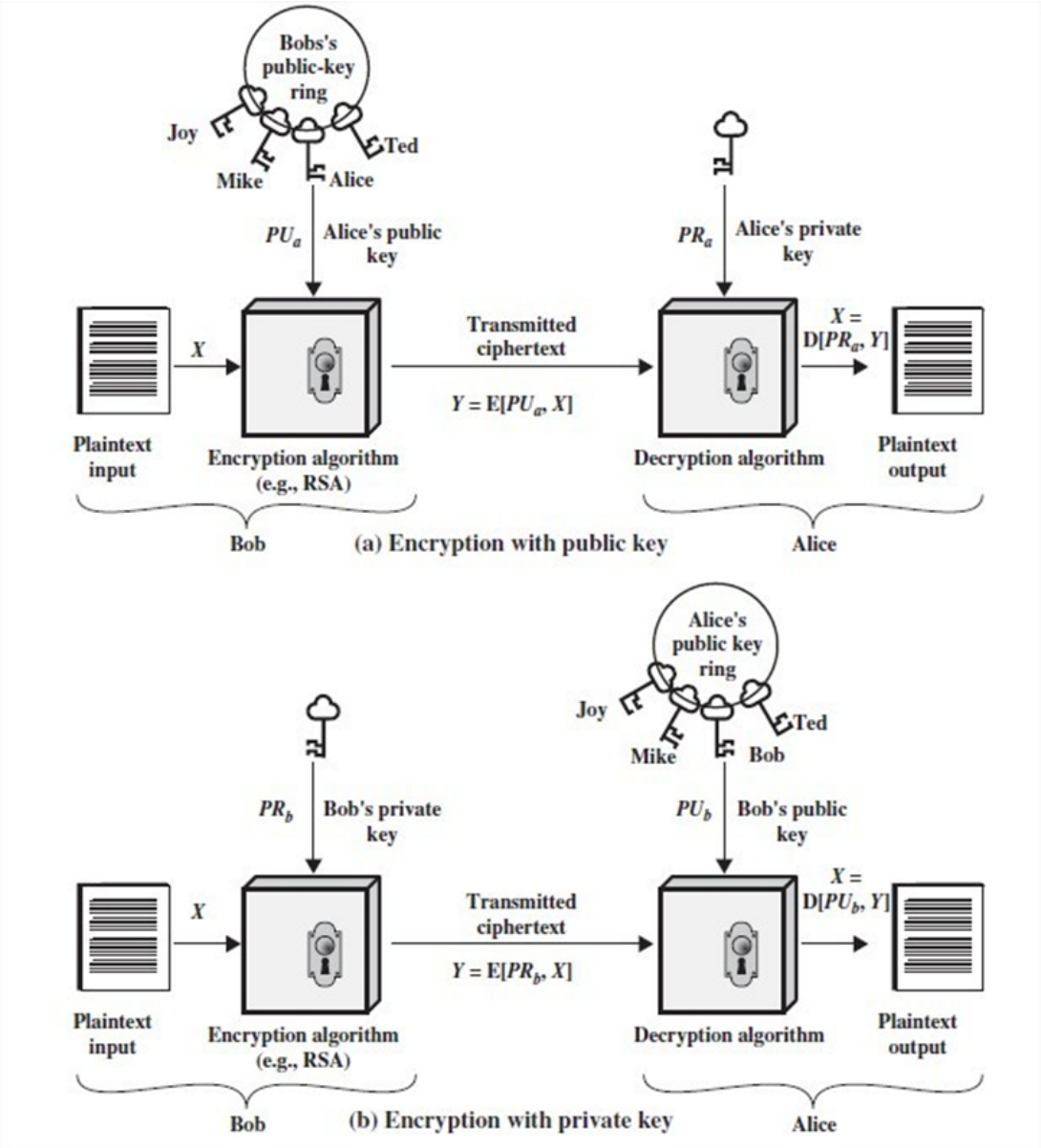
- Hệ mật mã khóa công khai cho tính xác thực (**Authentication**): Trong trường hợp này, Alice chuẩn bị một tin nhắn cho Bob và mã hóa nó bằng khóa riêng tư của Alice trước khi gửi đi. Bob có thể giải mã tin nhắn bằng khóa công khai của Alice. Bởi vì tin nhắn được mã hóa bằng khóa riêng tư của Alice, chỉ có Alice mới có thể đã chuẩn bị tin nhắn đó. Do đó, toàn bộ tin nhắn được mã hóa đóng vai trò như một chữ ký số (digital signature).
  - Mã hóa: C = E(M, PR)
  - Giải mã: M = D(C, PU)

1.3 Môi trường thực hành

Bài lab này yêu cầu bạn chuẩn bị các công cụ và môi trường sau để thực hiện các bài tập lập trình và phân tích mật mã:

- **Hệ điều hành:** Sử dụng máy tính chạy hệ điều hành Windows, Linux hoặc macOS có cài đặt môi trường Python 3.x .

- **Thư viện lập trình:** Cài đặt các thư viện mật mã và hỗ trợ tính toán thông qua



Hình 5: Two Approaches in Public-Key Cryptography

lệnh:

- `pip install pycryptodome sympy numpy matplotlib`

- **Công cụ chỉnh sửa Hex:** Cài đặt một trình chỉnh sửa mã hex như *Bless Hex Editor* hoặc *Ghex* để thực hiện việc can thiệp lỗi bit trực tiếp trên tệp tin dữ liệu .
- **IDE:** Sử dụng một môi trường phát triển tích hợp tùy chọn như Visual Studio Code, PyCharm hoặc bất kỳ trình soạn thảo mã nguồn nào hỗ trợ Python .

1.4 Chuẩn bị môi trường

Để đảm bảo mọi thứ diễn ra suôn sẻ và đạt được kết quả tốt nhất trong bài thực hành này, sinh viên cần thực hiện các chuẩn bị sau:

- **Kiến thức nền tảng:** Nắm vững các khái niệm cốt lõi về mật mã học hiện đại, bao gồm mật mã khối (DES, AES), mật mã khóa công khai (RSA), các chế độ hoạt động (ECB, CBC, CFB, OFB) và kỹ thuật đệm dữ liệu (Padding) .
- **Kỹ năng lập trình:** Có đủ kiến thức lập trình Python để thực hiện các nhiệm vụ như xử lý luồng byte, chuyển đổi định dạng (Hex, Binary, Base64) và triển khai các thuật toán dựa trên khung mã nguồn (scaffold code) được cung cấp .
- **Tìm hiểu thuật toán:** Cần đọc kỹ các định lý về cấu trúc mật mã Feistel (Định lý 2.2) và quy trình tạo khóa trong RSA để có thể tự tay lập trình mô phỏng các hệ mật mã này một cách chính xác .
- **Tài liệu mẫu:** Sinh viên nên tải trước mẫu báo cáo thực hành định dạng `.pdf` từ trang web UIT Courses và làm quen với định dạng nén tệp `.zip` theo yêu cầu nộp bài .

2 Nội dung thực hành

2.1 Cấu trúc Feistel và Sự lan truyền thay đổi

Trong bài tập này, bạn tự triển khai một hệ mật mã Feistel đơn giản để quan sát cách một thay đổi nhỏ ở đầu vào lan truyền qua từng vòng mã hóa.

Nhiệm vụ 2.1 (*)

Dựa trên Định lý 1.2, hãy hoàn thiện hàm `feistel_round`. Sau đó, thực hiện mã hóa hai bản rõ $M_1 = 0xAB$ và $M_2 = 0xAC$ (chỉ khác nhau 1 bit cuối).
Yêu cầu: In giá trị (L, R) sau mỗi vòng của cả hai quá trình và nhận xét về sự khác biệt giữa chúng tăng dần qua từng vòng như thế nào.

Gợi ý

Công thức mỗi vòng: $LE_i = RE_{i-1}$ và $RE_i = LE_{i-1} \oplus F(RE_{i-1}, K_i)$. Với F là hàm nội bộ kết hợp XOR và dịch bit [2].

Scaffold code:

```
def F(right, subkey):
    return (right ^ subkey) & 0x0F

def feistel_round(L_in, R_in, subkey):
    # TODO: Triển khai LE_i = RE_{i-1} và RE_i = LE_{i-1} XOR F(...)
```

```
def track_avalanche(msg, key):
    # L, R = (msg >> 4) & 0x0F, msg & 0x0F
    # subkeys = [key & 0x0F, (key >> 4) & 0x0F, (key + 1) & 0x0F, (key + 2) & 0x0F]

    print(f"Khởi tạo: L={format(L, f'0{4}b')}, R={format(R, f'0{4}b')}")
    for i in range(4):
        L, R = feistel_round(L, R, subkeys[i])
        print(f"Vòng {i+1}: L={format(L, f'0{4}b')}, R={format(R, f'0{4}b')}")
    return (L << 4) | R

# Chạy thử với 2 bản rõ khác nhau 1 bit
print("--- Mã hóa M1 ---")
track_avalanche(0xAB, 0x12)
print("--- Mã hóa M2 ---")
track_avalanche(0xAC, 0x12)
```

2.2 Mode of Operation - Chế độ hoạt động

Mục tiêu của bài tập này là giúp sinh viên phân biệt sự khác biệt về tính bảo mật giữa chế độ ECB và CBC khi xử lý các khối dữ liệu lặp lại.

Nhiệm vụ 2.2 (*)

Sử dụng thư viện `pycryptodome`, hãy viết script mã hóa chuỗi văn bản có tính lặp lại cao: `"UIT_LAB_UIT_LAB_UIT_LAB_UIT_LAB_"`. Thực hiện mã hóa bằng cả hai chế độ AES-ECB và AES-CBC (với IV cố định).

- In kết quả dưới dạng Hexadecimal và so sánh các khối 16-byte của bản mã.
- Bạn hãy tìm hiểu, thử nghiệm tất cả các Mode of Operation khác, và đưa ra nhận xét.

Lưu ý

Trong chế độ ECB, các khối bản rõ giống nhau sẽ cho ra các khối bản mã giống nhau, dẫn đến việc lộ diện cấu trúc dữ liệu.

Scaffold code:

```
from Crypto.Cipher import AES

key = b'1234567890123456'
plaintext = b"UIT_LAB_UIT_LAB_UIT_LAB_UIT_LAB_" # 32 bytes (2 blocks)
```

```
# 1. AES-ECB
cipher_ecb = AES.new(key, AES.MODE_ECB)
ct_ecb = cipher_ecb.encrypt(plaintext)

# 2. AES-CBC
#
#

print(f"ECB: {ct_ecb.hex()}")
print(f"CBC: {ct_cbc.hex()}")
```

2.3 Avalanche Effect

Hiệu ứng thác đổ đảm bảo rằng một thay đổi nhỏ ở đầu vào sẽ dẫn đến thay đổi lớn ở đầu ra, làm cho mật mã trở nên an toàn trước các phân tích sai biệt.

Nhiệm vụ 2.3 (*)

Viết script mã hóa hai chuỗi `"STAYHOME"` và `"STAYHOMA"` bằng thuật toán DES. Chuyển kết quả bản mã sang dạng chuỗi nhị phân và đếm số lượng bit khác nhau (Hamming Distance). Thay đổi key lần lượt với mã số sinh viên trong nhóm, tính toán tỷ lệ phần trăm thay đổi bit với các lần thay đổi và đưa ra nhận xét.

Gợi ý

Sử dụng hàm `bin(int.from_bytes(data, 'big'))` để chuyển đổi dữ liệu bytes sang chuỗi bit trong Python.

Scaffold code:

```
from Crypto.Cipher import DES

def avalanche_test():
    key = b'87654321'
    p1 = b'STAYHOME'
    p2 = b'STAYHOMA' # Chỉ khác 1 ký tự cuối so với p1

    # TODO: Thực hiện mã hóa p1, p2 bằng DES-ECB
    # TODO: Đếm số bit khác nhau giữa hai bản mã thu được
    # TODO: Tính tỷ lệ % bit bị thay đổi
```

Nhiệm vụ 2.4

SEED Lab - Secret-Key Encryption Lab (Task 5):
Sinh viên thực hiện các bước sau bằng Python để hiểu rõ về đặc tính lan truyền lỗi (error propagation) của các chế độ mã hóa khác nhau:

1. Tạo dữ liệu 1000 byte.
2. Mã hóa dữ liệu bằng AES-128.
3. Làm hỏng bản mã bằng cách đảo 1 bit tại byte thứ 26.
4. Giải mã bản mã đã bị lỗi và *quan sát có bao nhiêu khối dữ liệu bản rõ bị hỏng với chế độ mã hóa* được sử dụng lần lượt là ECB, CBC, CFB, hoặc OFB.

Nhiệm vụ 2.5

Tìm hiểu và so sánh độ mạnh và sự khác biệt của DES, Triple-DES (3DES), và AES. Tại sao chúng ta không nên sử dụng mã hóa double-DES (2DES)?

2.4 Mersenne Prime - Số nguyên tố lớn

Trước khi bắt đầu với thuật toán mã hóa khóa công khai, chúng ta sẽ ôn lại một số kiến thức lý thuyết số, cụ thể là kiểm tra số nguyên tố, ước chung lớn nhất (GCD) và phép toán modulo.

Nhiệm vụ 2.6 (*)

Viết một chương trình (bằng ngôn ngữ lập trình của bạn) để đáp ứng các yêu cầu sau:

1. Số nguyên tố:
   - Tạo ra một số nguyên tố ngẫu nhiên có 8 bits, 16 bits, 64 bits
   - Xác định 10 số nguyên tố lớn nhất nhỏ hơn số nguyên tố Mersenne thứ 10.
   - Kiểm tra xem một số nguyên tùy ý nhỏ hơn $2^{89} - 1$ có phải là số nguyên tố hay không?
2. Xác định ước số chung lớn nhất (GCD) của 2 *số nguyên lớn* tùy ý (lớn nhất có thể mà bạn có thể xử lý).
3. Tính toán lũy thừa theo module $a^x \bmod p$. Chương trình của bạn phải có khả năng tính toán trong trường hợp số mũ lớn (x>40), ví dụ, $7^{40} \bmod 19$

Gợi ý

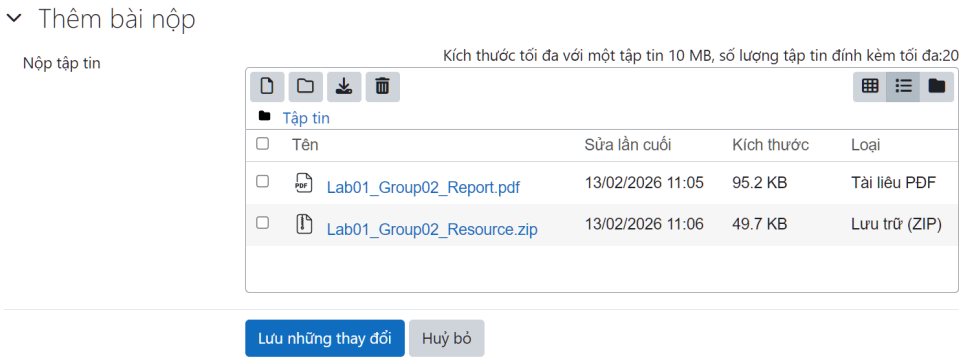
- Để kiểm tra các số nguyên tố lớn (Mersenne Prime), bạn có thể tìm hiểu và sử dụng thuật toán Miller-Rabin.
- Để xác định ước số chung lớn nhất (greatest common divisor), bạn nên sử dụng thuật toán Euclid.
- Phép nhân module có tính chất sau:

$$(a \times b) \equiv [(a \bmod n) \times (b \bmod n)] \bmod n$$

  Tính chất này có thể được áp dụng để tính toán lũy thừa module (modular exponentiation) trong trường hợp có số mũ lớn.

3 Yêu cầu và đánh giá

Sinh viên được khuyến khích làm việc theo nhóm từ hai đến ba người.



Hình 6: Yêu cầu bài nộp trên hệ thống UIT Courses

Yêu cầu chung

Bài nộp phải tuân thủ các quy định sau:

- **Bài nộp phải được nộp trên hệ thống UIT Courses** (như hình 6), bao gồm:
  1. Một tệp báo cáo thực hành ở định dạng `LabXGroupY_Report.pdf`, trình bày theo đúng mẫu quy định.
  2. Một tệp `LabXGroupYY_Resource.zip` (nếu có), bao gồm mã nguồn, dữ liệu, chương trình thực thi và các tài nguyên liên quan phục vụ cho việc kiểm tra và đánh giá.
- **Báo cáo thực hành** cần đảm bảo những yêu cầu sau:

1. Báo cáo phải trình bày đầy đủ nội dung thực hiện, phương pháp, kết quả và các nhận xét cần thiết; nội dung phải rõ ràng, chính xác và có tính hệ thống và hình ảnh minh chứng cần thiết.
2. Báo cáo không chấp nhận việc sử dụng đồng thời nhiều ngôn ngữ trong cùng một báo cáo, trừ các thuật ngữ kỹ thuật hoặc từ khóa chuyên ngành (cả một câu, cả một đoạn văn).
3. Đối với các nhiệm vụ lập trình, báo cáo phải mô tả chi tiết các thành phần chính của chương trình, giải thích các bước thực hiện và minh họa kết quả chạy chương trình khi cần thiết. Việc chỉ nộp mã nguồn mà không có mô tả hoặc phân tích sẽ không được chấm điểm cho nhiệm vụ đó.
4. Bài nộp phải là kết quả làm việc của chính sinh viên hoặc nhóm sinh viên. Sinh viên được phép thảo luận để trao đổi ý tưởng, nhưng mọi hình thức sao chép toàn bộ hoặc một phần báo cáo đều bị xem là vi phạm quy định về liêm chính học thuật và không được đánh giá điểm cho bài thực hành. Mọi tài liệu tham khảo phải được trích dẫn đầy đủ.

- **Nhóm sinh viên phải sử dụng một kho mã nguồn GitHub trong suốt quá trình thực hiện các bài lab.** Các yêu cầu cụ thể như sau:
  1. Mỗi bài lab phải được lưu trữ trong kho mã nguồn GitHub và được cập nhật sau khi hoàn thành.
  2. Mã nguồn phải được tổ chức theo thư mục tương ứng với từng lab, có cấu trúc rõ ràng và nhất quán.
  3. Mỗi thư mục lab phải kèm theo tệp `README.md` mô tả ngắn gọn mục tiêu, nội dung chính và hướng dẫn chạy chương trình (nếu có).
  4. Kho mã nguồn phải chứa đầy đủ các phiên bản mã nguồn đã thực hiện trong học phần và duy trì ở trạng thái có thể truy cập được (public hoặc cấp quyền cho giảng viên) để phục vụ việc kiểm tra khi cần thiết.

Lưu ý

Việc đánh giá được thực hiện dựa trên các tiêu chí sau:

- Mức độ hoàn thành của từng nhiệm vụ theo yêu cầu đề bài.
- Tính đúng đắn của kết quả và phương pháp thực hiện.
- Chất lượng trình bày trong báo cáo (rõ ràng, logic, đầy đủ minh họa khi cần).
- Chất lượng mã nguồn (tổ chức, khả năng chạy lại, tính rõ ràng).
- Mức độ tuân thủ các quy định về nộp bài và quản lý mã nguồn.

Đánh giá

- Task 2.1 được tính 1 điểm.
- Các Task 2.2, 2.3, 2.4 và 2.6, mỗi task được tính 1.5 điểm.
- Task 2.5 được tính 2 điểm (lưu ý về việc sử dụng AI).
- Các task có dấu (*) là những task được thực hiện tại lớp; nếu nộp bổ sung thì điểm tối đa chỉ đạt 70%.

Lưu ý: Không chia sẻ bất kỳ tài liệu nào của học phần (slide bài giảng, tài liệu đọc, bài tập, bài lab, v.v.) ra ngoài lớp học khi chưa có sự cho phép của giảng viên.

Chúc các bạn học tập tốt và đạt kết quả cao trong học phần!!!